

string_view range constructor should be explicit

Document number: P2499R0
Date: 2021-12-07
Project: Programming Language C++
Audience: LEWG
Reply-to: James Touton <bekenn@gmail.com>

Introduction

[P1989R2](#) added a new constructor to `basic_string_view` that allows for implicit conversion from any contiguous range of the corresponding character type. This implicit conversion relies on the premise that a range of `char` is inherently string-like. While that premise holds in some situations, it is hardly universally true, and the implicit conversion is likely to cause problems. This paper proposes making the conversion explicit instead of implicit in order to avoid misleading programmers.

Rationale

[P1391R3](#) (a precursor to [P1989R2](#)) justifies making the conversion implicit with the incorrect notion that "a contiguous range of `character[s]` is the same platonic thing as a `string_view`", despite correctly pointing out that "[ranges] with different [traits types] should not be implicitly convertible". The latter acknowledgment recognizes that there are semantic nuances here beyond the value type, and as a result, no direct conversion is provided from range types having a mismatched `traits_type`.

One such semantic difference between a string and an arbitrary range of `char` is mentioned in [P1391R3](#) (lightly modified for correctness):

```
char const t[] = "text";  
std::string_view s1(t); // s1.size() == 4;  
  
std::span<char const> tv(t);  
std::string_view s2(tv); // s2.size() == 5;
```

Here, `s1` and `s2` are constructed from equivalent ranges of `const char`, but the resulting `string_view` objects are different. This is because overload resolution for the array argument selects `string_view`'s constructor from `const char*`, a type which by convention points to a string followed by a null terminator. The terminator is not semantically part of the string, so the resulting `string_view` doesn't include it. The `span`, by contrast, does include the null terminator.

Laudably, [P1989R2](#) recognizes several mechanisms by which a type may indicate that it provides string-like data, and the range constructor is disabled in these cases:

- The type is implicitly convertible to `const charT*`
- The type provides its own conversion function to the target `basic_string_view` specialization
- The type defines its own `traits_type`, and that type differs from the string view's `traits_type`

The presence of these mechanisms refutes the notion that "a contiguous range of `character[s]` is the same platonic thing as a `string_view`". Nonetheless, it is certainly true that constructing a `string_view` from a range of `char` is a useful operation, provided that the user knows that the entire range actually constitutes a string. This paper therefore proposes to keep the range constructor, but make it explicit.

Pitfalls

Very often, a contiguous range of `char` is used as a buffer for storing string data. This does not imply that the *entire* range constitutes a string:

```
extern void get_string(std::span<char> buffer);
extern void use_string(std::string_view str);

char buf[200];
get_string(buf);
use_string(buf);
```

This code is representative of quite a lot of real-world code that exists today. The `get_string` function fills a portion of a buffer with a null-terminated string, and the `use_string` function consumes that string. This code works in C++20, and would also work in C++17 with a minor modification to `get_string` to pass the buffer as a pointer and size instead of as a span. This code will continue to work in the presence of [P1989R2](#); the range constructor is disabled because the array is convertible to `const char*` (and even if it weren't disabled, overload resolution would prefer the `const char*` constructor anyway).

Many code style guidelines emphasize the use of `std::array` over raw arrays, so let's make that change:

```
extern void get_string(std::span<char> buffer);
extern void use_string(std::string_view str);

std::array<char, 200> buf;
get_string(buf);
use_string(buf); // oops
```

The code compiles and runs, and in many cases will appear to work, but where the length of the `string_view` parameter used to be inferred from the presence of a null terminator, it is now unavoidably the size of the entire buffer, and unquestionably wrong given that the prior code was correct. If the range constructor were explicit, this code would generate an error diagnostic.

The same sort of thing can easily happen with vectors. For instance, an API might require the user to invoke a function that provides an estimate for a buffer size, which the user then allocates before calling another function that fills the buffer. The estimate may return a size greater than that actually needed by the resulting string if calculating the exact size would be expensive:

```
extern size_t estimate_string_size();
extern void get_string(std::span<char> buffer);
extern void use_string(std::string_view str);

size_t estimated_size = estimate_string_size();
std::vector<char> buf(estimated_size);
get_string(buf);
use_string(buf); // oops
```

[P1391R3](#) states: "We think this proposed design is consistent with existing practices of having to be explicit about the size in the presence of embedded nulls[.]" This paper respectfully disagrees.

Design decisions

The intent of [P1989R2](#) is to allow for conversion from a range to a string view. LEWG has already decided that this is a good idea, and this paper concurs. Removing the range constructor would be counter-productive, but keeping it in its current form is also problematic. That leaves us with a couple of options.

Option 1: Make the range constructor explicit

This is the preferred approach of this paper. This approach preserves the functionality gains offered by [P1989R2](#) while making it harder to invoke the conversion by accident. Users who know that the source range actually represents a string can still take advantage of the conversion. Consider the vector example above, but with `get_string` modified to return the number of characters written to the buffer:

```
extern size_t estimate_string_size();
extern size_t get_string(std::span<char> buffer);
extern void use_string(std::string_view str);

size_t estimated_size = estimate_string_size();
std::vector<char> buf(estimated_size);
size_t actual_size = get_string(buf);
buf.resize(actual_size);
use_string(std::string_view(buf)); // ok
```

Option 2: Make the range constructor conditionally explicit

If the source type defines its own `traits_type`, and that type is the same as the string view's `traits_type`, then the source range can reasonably be assumed to represent a string. This appears to be a good approach, but does add a small amount of complexity to the specification and may be a more difficult rule to teach than Option 1. This paper is not opposed to Option 2.

Option 3: Make the range constructor (conditionally) explicit and remove the `traits_type` constraint

This modifies either Option 1 or Option 2 by additionally removing the constraint that the source range's `traits_type` (if present) must match the string view's `traits_type`. Given that the constructor is already explicit, the user is already primed to expect that the resulting string view is not semantically equivalent to the source range in every respect. Moreover, the name `traits_type` is somewhat generic; there's nothing in that name that implies the traits are string traits.

This change would allow for explicit conversion from a string or string view with dissimilar traits. This paper agrees with P1391R3 that "strings with different [traits types] should not be implicitly convertible", but an *explicit* conversion may be sensible. This paper does not attempt to explore the consequences of this design, and so this approach is not recommended.

Wording

All modifications are presented relative to [N4901](#).

Modify §21.4.3.1 [\[string.view.template.general\]](#) and the corresponding heading prior to §21.4.3.2 [\[string.view.cons\]](#) paragraph 11:

```
template<class R>
    constexpr explicit basic_string_view(R&& r);
```

References

1. [\[P1391R3\]](#) Corentin Jabot; "Range constructor for `std::string_view`"
2. [\[P1989R2\]](#) Corentin Jabot; "Range constructor for `std::string_view` 2: Constrain Harder"
3. [\[N4901\]](#) "Working Draft, Standard for Programming Language C++"